

QuickDelivery - App Flutter

1. Objetivo

O app Flutter unificado implementa os fluxos do cliente e do entregador. Ele permite que um usuário com papel `CUSTOMER` faça login, consulte suas entregas, crie novas solicitações, acompanhe detalhes e cancele entregas quando a máquina de estados permitir. Usuários com papel `DELIVERYMAN` conseguem visualizar entregas pendentes, aceitar demandas, iniciar a execução, concluir entregas, cancelar entregas aceitas atribuídas a eles e consultar histórico.

2. Telas

Login

A tela de login consome `POST /auth/login`, mantém o token em memória durante a sessão e direciona o usuário conforme o papel retornado pelo backend.

Minhas Entregas

A tela principal do cliente consome `GET /deliveries` e lista apenas as entregas do cliente autenticado, conforme a autorização do backend. Cada card mostra descrição, origem, destino, status e data de atualização. A tela possui ação manual de atualizar, acesso ao perfil e botão para criar nova entrega.

Nova Entrega

A tela de criação envia `POST /deliveries` com `customerId`, `pickupAddress`, `dropoffAddress` e `description`. Após criação bem-sucedida, o app abre o detalhe da entrega criada.

Detalhes da Entrega

A tela de detalhe consome `GET /deliveries/:id`, exibe status, origem, destino, descrição, entregador atribuído e datas. Para clientes, a tela permite cancelar entregas em `PENDING` ou `ACCEPTED`. Para entregadores, a mesma tela permite aceitar entregas pendentes, iniciar entregas aceitas, concluir entregas em andamento e cancelar entregas aceitas atribuídas ao entregador autenticado.

Perfil

A tela de perfil exibe dados do cliente autenticado e oferece logout com confirmação.

Área do Entregador

A área do entregador possui abas para entregas disponíveis, minhas entregas e histórico. A aba de disponíveis mostra entregas `PENDING` e permite aceitar uma demanda. A aba de minhas entregas mostra entregas `ACCEPTED` e `IN_PROGRESS`, permitindo iniciar ou concluir a execução. A aba de histórico exibe entregas `DELIVERED` e `CANCELLED`.

3. Atualização Assíncrona

A atualização assíncrona foi implementada por polling, alternativa permitida no enunciado da Sprint 3. A lista de entregas, a tela de detalhe e a área do entregador consultam o backend automaticamente a cada 5 segundos, permitindo que mudanças feitas por outro usuário sejam refletidas sem ação manual.

Na lista e na área do entregador, o polling chama `GET /deliveries` enquanto o usuário está autenticado. No detalhe, o polling chama `GET /deliveries/:id` enquanto a entrega ainda não está em estado final. O usuário também pode atualizar manualmente com pull-to-refresh ou pelo botão de atualização.

4. Arquitetura

```
mobile_app/lib/
├─ config/          # URL base da API
├─ controllers/     # estado da sessão e entregas
├─ models/          # User, AuthSession e Delivery
├─ screens/         # Login, cliente, entregador, detalhe, criação e perfil
├─ services/        # cliente HTTP e serviços REST
├─ theme/           # tema visual do app
├─ utils/           # helpers de formatação
└─ widgets/        # componentes reutilizáveis
```

Responsabilidades principais:

- `ApiConfig` define a URL base por `--dart-define=QUICKDELIVERY_API_URL`.
- `ApiClient` centraliza chamadas HTTP, JSON, token Bearer e tratamento de erro.
- `AuthService` isola autenticação.
- `DeliveriesService` isola listagem, criação, consulta e transições de status.
- `AppController` mantém sessão, token, lista de entregas, loading, erros e ações de cliente/entregador.
- `screens` implementam navegação e telas do cliente/entregador.
- `models` convertem os payloads JSON da API para objetos Dart.

Fluxo do cliente:

```
Login -> Minhas Entregas -> Nova Entrega -> Detalhes da Entrega
                                -> Detalhes da Entrega
                                -> Perfil -> Logout
```

Fluxo do entregador:

```
Login -> Área do Entregador -> Disponíveis -> Aceitar
                                -> Minhas -> Iniciar -> Concluir
                                -> Histórico
                                -> Detalhes da Entrega
```

5. Execução

As instruções completas de execução estão no `README.md` da raiz do projeto e no `mobile_app/README.md`. Esses arquivos descrevem como subir o backend, iniciar o emulador Android, configurar `QUICKDELIVERY_API_URL` e rodar o app.

Credenciais seedadas para teste:

Perfil	Email	Senha
Cliente	<code>customer1@example.com</code>	<code>password123</code>
Entregador	<code>deliveryman1@example.com</code>	<code>password123</code>

6. Aderência às Sprints 3 e 4

Requisito do enunciado	Implementação no projeto
App Flutter funcional para o cliente	Código em <code>mobile_app</code> , validado em emulador Android.
Mínimo de 3 telas	Login, Minhas Entregas, Nova Entrega, Detalhes da Entrega, Perfil e Área do Entregador.
Integração com backend REST	Consome <code>POST /auth/login</code> , <code>GET /deliveries</code> , <code>POST /deliveries</code> , <code>GET /deliveries/:id</code> e <code>PATCH /deliveries/:id/status</code> .
Atualização assíncrona de estado	Polling automático a cada 5 segundos na lista, no detalhe e na área do entregador.
Arquitetura documentada	Separação em <code>models</code> , <code>services</code> , <code>controllers</code> , <code>screens</code> , <code>widgets</code> , <code>theme</code> , <code>utils</code> e <code>config</code> .
Código-fonte executável	App roda no emulador Android com <code>QUICKDELIVERY_API_URL=http://10.0.2.2:3000</code> .
Fluxo operacional do entregador	Área do entregador com abas, aceite, início, conclusão, cancelamento permitido e histórico.